1.What do you mean by an Operating System?
Explain the components of an Operating System, its basic objectives, and discuss the features, advantages, and limitations of Mobile Operating Systems.
Ans
The Operating System (OS)
An Operating System is a collection of software that manages computer hardware resources and provides common services for computer programs. It acts as an intermediary (interface) between the user and the computer hardware.
Basic Objectives
• Convenience: To provide a user-friendly interface (GUI or CLI) that makes operating the computer straightforward.
• Efficiency: To act as a resource manager, ensuring that the CPU, memory, and I/O devices are utilized to their maximum potential.
• Throughput: To increase the total amount of work (jobs) the system can perform in a given time period.
• Security: To protect the system's data and resources from unauthorized access and ensure process isolation.
Components of an OS:
• The Kernel: The central core of the OS. It is the first program loaded and has complete control over everything in the system. It handles the transition between user-mode and kernel-mode.
• Process Management: Manages the life cycle of processes—from creation and scheduling to execution and termination. It also handles Inter-Process Communication (IPC).
• Memory Management: Responsible for the allocation and deallocation of primary memory (RAM). It keeps track of which parts of memory are currently in use and by whom.
• File Management: Provides a uniform logical view of information storage. It manages the creation, deletion, and manipulation of files and directories.
• Device Management: Communicates with hardware via Device Drivers. It masks the peculiarities of specific hardware devices from the user.
Mobile Operating Systems:
• A Mobile Operating System is specialized software that manages the hardware of smartphones and tablets. It is optimized for portability, battery efficiency, and touch-based input.
Features
• Touch Interface: Optimized for finger gestures (swipe, pinch, tap) instead of a mouse.
• Power Management: Aggressively manages background tasks to preserve battery life.
• Sensor Integration: Built-in support for GPS, accelerometers, and biometrics.

• Connectivity: Seamless switching between Cellular (5G/4G) and Wi-Fi.

Advantages

• Portability: High-performance computing in a pocket-sized form factor.
• Ease of Use: Intuitive, icon-based interfaces accessible to all users.
• Instant Access: “Always-on” functionality with real-time notifications.

Limitations

• Resource Constraints: Lower RAM and CPU power compared to desktops.
• Thermal Limits: No cooling fans; the system must slow down (throttle) if it gets too hot.
• Screen Size: Limited “real estate” makes complex multitasking difficult.

2.Explain the types of Operating Systems.
What is the difference between a loosely coupled system and a tightly coupled system?
Ans

Types of Operating Systems

• Batch OS: Jobs with similar needs are grouped and executed as a group. Interaction between the user and the computer is minimal during execution.
• Time-Sharing OS: Uses CPU scheduling and multi-programming to provide each user with a small portion of a shared computer at once.
• Real-Time OS (RTOS): Systems where the time interval required to process and respond to inputs is so small that it controls the environment (e.g., air traffic control).
• Distributed OS: Manages a group of distinct computers and makes them appear to be a single computer.
• Network OS: Runs on a server and provides the capability to manage data, users, groups, security, and applications over a network.

Tightly Coupled Systems (Parallel Processing)

In these systems, multiple processors share the same primary memory and a system clock.

• Communication: Occurs through the shared memory.
• Connectivity: Processors are connected via a high-speed system bus.
• Control: Usually managed by a single operating system.
• Efficiency: Very high data transfer rates, but limited scalability because the bus becomes a bottleneck if too many CPUs are added.

Loosely Coupled Systems (Distributed Systems)

Each processor has its own local memory. The processors are independent and communicate with each other through various communication lines.

• Communication: Occurs through message passing.
• Connectivity: Connected via a network (LAN/WAN) or high-speed links.
• Control: Each node/processor usually runs its own local operating system.

• Efficiency: Highly scalable and robust. If one node fails, the rest of the system remains operational (high fault tolerance).

3.CPU Scheduling Problems

a) Draw the Gantt chart for Round Robin scheduling (Time Quantum = 2) and calculate turnaround time, average waiting time, throughput, and processor utilization for:
P1: 5, P2: 3, P3: 1, P4: 2, P5: 3

b) Consider the following processes and compute average waiting time, response time, and turnaround time using FCFS scheduling:
P1: Arrival 0, Burst 8
P2: Arrival 2, Burst 4
P3: Arrival 4, Burst 6
P4: Arrival 6, Burst 2

Ans

a) Round Robin (Time Quantum = 2)

Processes: P1(5), P2(3), P3(1), P4(2), P5(3)
Gantt Chart:
| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P5 | P1 |
0 2 4 5 7 9 11 12 13 14
Calculations:
• P1: Finishes at 14. TAT = 14, WT = 14 – 5 = 9.
• P2: Finishes at 12. TAT = 12, WT = 12 – 3 = 9.
• P3: Finishes at 5. TAT = 5, WT = 5 – 1 = 4.
• P4: Finishes at 7. TAT = 7, WT = 7 – 2 = 5.
• P5: Finishes at 13. TAT = 13, WT = 13 – 3 = 10.
Averages:
• Avg Turnaround Time: (14+12+5+7+13) / 5 = 10.2
• Avg Waiting Time: (9+9+4+5+10) / 5 = 7.4
• Throughput: 5 processes / 14 units \approx \mathbf{0.35 \text{ processes/unit}}
• Processor Utilization: (14/14) \times 100 = \mathbf{100\%}

b) FCFS Scheduling (with Arrival Times)

Processes: P1(0,8), P2(2,4), P3(4,6), P4(6,2)
Execution Order:
• P1: Runs 0 to 8.
• P2: Runs 8 to 12.
• P3: Runs 12 to 18.
• P4: Runs 18 to 20.
Results Table:

Process	Arrival	Burst	Exit	TAT	WT	RT
P1	0	8	8	8	0	0
P2	2	4	12	10	6	6
P3	4	6	18	14	8	8
P4	6	2	20	14	12	12

Final Averages:

• Avg Waiting Time: (0+6+8+12) / 4 = \mathbf{6.5}

• Avg Turnaround Time: (8+10+14+14) / 4 = \mathbf{11.5}

• Avg Response Time: (0+6+8+12) / 4 = \mathbf{6.5}

4.Deadlock

Define deadlock. Explain the four necessary conditions for deadlock to occur. Discuss deadlock prevention and deadlock avoidance techniques.

Ans

A Deadlock is a specific condition in a multi-processing environment where a set of processes are blocked because each process is holding a resource and waiting for another resource held by another process in the same set. In this state, none of the processes can proceed, and the system hangs.

The Four Necessary Conditions

For a deadlock to occur, the following four conditions (known as Coffman Conditions) must hold simultaneously:

• Mutual Exclusion: At least one resource must be held in a non-sharable mode; only one process can use the resource at any given time.

• Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

• No Preemption: Resources cannot be forcibly taken from a process; they can only be released voluntarily by the process holding them after it has completed its task.

• Circular Wait: A set of processes \{P_0, P_1, \dots, P_n\} must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, and P_n is waiting for a resource held by P_0.

1. Deadlock Prevention

The goal of prevention is to ensure that at least one of the four necessary conditions listed above cannot hold.

• Attacking Mutual Exclusion: Generally difficult as some resources (like printers) are inherently non-sharable.

• Attacking Hold and Wait: Require processes to request all required resources at the start. If all are not available, the process gets none.

• Attacking No Preemption: If a process requesting a resource is denied, it must release all its currently held resources so other processes can use them.
• Attacking Circular Wait: Impose a linear ordering of resource types. Processes can only request resources in increasing order of priority.

2. Deadlock Avoidance

Avoidance requires the OS to have prior information about which resources a process will request during its lifetime. The system stays in a Safe State.
• Safe State: A state where the system can allocate resources to each process in some order and still avoid a deadlock.
• Banker's Algorithm: The most famous avoidance algorithm. It checks every resource request. If granting the request leads to an "unsafe state" (potential deadlock), the request is denied or delayed, even if the resource is currently free.

5.Disk Scheduling Algorithms

a) Explain FCFS and SCAN disk scheduling algorithms with suitable head-movement diagrams.
(Starting cylinder = 100, direction = increasing, requests: 150, 160, 194, 94, 56, 54, 38, 37, 18)

c) Explain and compute total head movement using SSTF and C-SCAN algorithms.
(Queue: 23, 89, 132, 42, 187; starting head = 100)

Disk scheduling is performed by the operating system to schedule I/O requests arriving for the disk. Since moving the disk head is a slow mechanical process, these algorithms aim to reduce the Total Head Movement (Seek Time).

Ans

a) FCFS and SCAN Algorithms

Scenario: Starting cylinder = 100, Direction = Increasing.
Requests: 150, 160, 194, 94, 56, 54, 38, 37, 18.

I. FCFS (First-Come, First-Served)

Requests are addressed in the exact order they arrive in the disk queue. It is fair but not efficient.
• Path: 100 – 150 – 160 – 194 – 94 – 56 – 54 – 38 – 37 – 18.
• Total Head Movement: |150-100| + |160-150| + |194-160| + |94-194| + |56-94| + |54-56| + |38-54| + |37-38| + |18-37| = 50 + 10 + 34 + 100 + 38 + 2 + 16 + 1 + 19 = 270 cylinders.

II. SCAN (Elevator Algorithm)

The head moves in one direction (increasing) to the very end of the disk, then reverses direction to service the remaining requests.
• Path: 100 – 150 – 160 – 194 – 199 -0- 94 – 56 – 54 – 38 – 37 – 18.
• Total Head Movement: (199 – 100) + (199 – 18) = 99 + 181 = 280cylinders.

b) SSTF and C-SCAN Algorithms

Queue: 23, 89, 132, 42, 187 | Starting Head: 100

I. SSTF (Shortest Seek Time First)

The algorithm selects the request closest to the current head position to minimize immediate seek time.

- From 100, closest is 89 (diff 11).
- From 89, closest is 132 (diff 43). Note: 42 is 47 units away.
- From 132, closest is 187 (diff 55).
- From 187, the head must turn back to 42 (diff 145).
- From 42, closest is 23 (diff 19).
- Path: 100 – 89 – 132 – 187 -42 – 23.
- Total Head Movement: 11 + 43 + 55 + 145 + 19 = 273 cylinders.

II. C-SCAN (Circular SCAN)

The head moves from one end to the other, servicing requests. When it reaches the end, it immediately returns to the beginning (cylinder 0) without servicing any requests on the return trip.

- Path: 100 – 132 – 187 – 199 – 0- 23 – 42 – 89.
- Total Head Movement: (199 – 100) + (199 – 0) + (89 – 0) = 99 + 199 + 89 = 387 cylinders.

6.Memory Management Techniques

a) Explain paging address translation using direct mapping and associative mapping with examples and diagrams.

b) What is segmentation? How does it solve the problems of contiguous memory allocation?

Ans

a) Paging Address Translation

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. This permits the physical address space of a process to be non-contiguous.

I. Direct Mapping

In direct mapping, the CPU generates a Logical Address, which is divided into a Page Number (p) and a Page Offset (d).

- The Page Number serves as an index into a Page Table.
- The Page Table stores the corresponding Frame Number (f) in physical memory (RAM).
- The physical address is then constructed by combining the Frame Number with the Offset.

II. Associative Mapping (TLB)

Standard paging requires two memory accesses (one for the page table, one for the data), which is slow. To speed this up, a Translation Lookaside Buffer (TLB) is used.

• The TLB is a high-speed associative cache that stores the most recently used page-to-frame mappings.

• TLB Hit: The CPU finds the mapping in the TLB instantly.

• TLB Miss: The CPU must access the page table in RAM, then updates the TLB for the next use.

c) Segmentation

Segmentation is a memory management technique that satisfies the "programmer's view" of memory. Instead of viewing memory as a single linear array, it is viewed as a collection of logical segments (e.g., Main Program, Stack, Library, Global Variables).

Solving Contiguous Memory Allocation Problems:

• Dynamic Placement: Segments of a single process can be placed in different parts of physical memory, so the OS doesn't need to find one massive contiguous block.

• Reduces External Fragmentation: While segmentation can still lead to external fragmentation, the segments are smaller than the entire process, making them easier to fit into available "holes."

• Growth of Data Structures: Data structures like stacks and heaps can grow or shrink independently within their own segments without overlapping other program parts.

• Sharing and Protection: It is easier to share a specific segment (like a shared library) between processes or protect a segment (making a code segment "Read-Only").

7.Process Management

a) What is Process Control Block (PCB)?

b) Explain context switching between two processes with steps and proper illustration.

Ans

a) A Process Control Block (PCB) is a data structure used by the Operating System to store all the information required to manage a specific process. Every time a process is created, the OS creates a corresponding PCB. It serves as the "identity card" for the process.

Key Information stored in a PCB:

• Process State: The current status of the process (New, Ready, Running, Waiting, or Terminated).

• Process ID (PID): A unique identification number assigned to each process.

• Program Counter (PC): Indicates the address of the next instruction to be executed for this process.

• CPU Registers: Temporary storage locations used by the CPU (accumulators, index registers, etc.) that must be saved during a switch.

• Memory Management Info: Information about page tables or segment tables.

• I/O Status Info: A list of I/O devices allocated to the process and a list of open files.
b) Context Switching is the mechanism of saving the current state (context) of a running process so that it can be paused and later resumed from the same point, while another process is given control of the CPU.
Steps in Context Switching:
• Interrupt Occurs: The OS receives a signal (like a timer interrupt or I/O request) to stop the current process (P_1).
• Save State: The current values of the CPU registers and Program Counter for P_1 are saved into its PCB (PCB_1).
• Update State: The status of P_1 is changed from "Running" to "Ready" or "Waiting."
• Select Next Process: The scheduler selects a new process (P_2) to execute.
• Reload State: The OS loads the state of P_2 from its PCB (PCB_2) into the CPU registers.
• Resume Execution: The CPU starts executing P_2 from the address stored in its Program Counter.

8.File System
a) What do you mean by a file? Explain the various ways of accessing a file.
b) Explain the three file allocation methods with suitable diagrams.
Ans
a) A file is a named collection of related information that is recorded on secondary storage (such as a hard disk or SSD). From a user's perspective, a file is the smallest allotment of logical secondary storage.
Ways of Accessing a File:
• Sequential Access: Information in the file processed in order, one record after the other. This is common for editors and compilers (e.g., magnetic tapes).
• Direct (Random) Access: The file is viewed as a numbered sequence of blocks or records. A program can read or write any block immediately by its block number, without needing to go through previous blocks.
• Indexed Access: An index is created which contains pointers to various blocks. To find a record, you first search the index and then use the pointer to access the file directly.
b) File Allocation Methods
Allocation methods define how disk blocks are allocated for files. The three primary methods are:

1. Contiguous Allocation

Each file occupies a set of adjacent blocks on the disk.
• Pros: Excellent performance (minimal head movement); supports both sequential and direct access.
• Cons: Leads to external fragmentation; difficult to find space for a growing file.

2. Linked Allocation

Each file is a linked list of disk blocks. The blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks.

• Pros: No external fragmentation; files can grow easily.

• Cons: Effective only for sequential access; if a pointer is lost or corrupted, the rest of the file is inaccessible.

3. Indexed Allocation

All the pointers for a file are brought together into one location called the Index Block. Each file has its own index block, which contains the disk block addresses for that file.

• Pros: Supports efficient direct access; no external fragmentation.

• Cons: Suffers from pointer overhead (even small files need an entire index block).

9.Schedulers in Operating System

a) What is a scheduler? Explain the types of schedulers.

b) Give one-word answers:

(i) Module that gives control of CPU to selected process

(ii) Scheduler controlling degree of multiprogramming

(iii) Scheduler responsible for ready → running state

(iv) Scheduler responsible for suspending and resuming processes

Ans

a) A Scheduler is a specialized OS module that manages the selection and removal of processes from various queues (Job Queue, Ready Queue, and Device Queue). The main goal is to maximize CPU utilization and balance the system load.

Types of Schedulers:

• Long-term Scheduler (Job Scheduler): * Determines which programs are admitted to the system for processing.

• It selects processes from the disk (job pool) and loads them into memory for execution.

• Goal: To maintain a good mix of I/O-bound and CPU-bound processes to control the degree of multiprogramming.

• Short-term Scheduler (CPU Scheduler): * Selects a process from among the processes that are ready to execute and allocates the CPU to it.

• It runs very frequently (often every few milliseconds).

• Goal: To optimize system performance based on criteria like throughput and response time.

• Medium-term Scheduler: * It is responsible for swapping processes out of memory to disk and vice versa.

• If the memory becomes overcrowded, it reduces the degree of multiprogramming by moving a process from memory to the “swap space” on the disk.

• Goal: To balance the memory load and free up RAM for other processes.

b) One-Word Answers

(i) Module that gives control of CPU to selected process: Dispatcher

(ii) Scheduler controlling degree of multiprogramming: Long-term Scheduler

(iii) Scheduler responsible for ready \rightarrow running state: Short-term Scheduler

(iv) Scheduler responsible for suspending and resuming processes: Medium-term Scheduler

1. Define Kernel of an Operating System.

The Kernel is the heart of the operating system. It is the first program loaded into memory when the system boots and remains active until shutdown. It acts as the primary interface between the hardware and the applications, managing CPU time, memory, and peripheral devices. It operates in a protected "Kernel Mode" to prevent user applications from crashing the entire system.

2. Define First Fit, Best Fit, and Worst Fit memory allocation.

These are strategies for Dynamic Storage Allocation:

• First Fit: The OS allocates the first available hole that is large enough for the process. It is the fastest strategy but can lead to a cluttered memory start.

• Best Fit: The OS searches the entire list to find the smallest hole that is big enough. It produces the smallest leftover hole, aiming to use memory efficiently, though it often leaves tiny, unusable gaps.

• Worst Fit: The OS allocates the largest available hole. The logic is that the leftover space will be large enough to be useful for another process.

3. What is trap addressing in segmentation?

In a segmented memory system, every logical address is a pair (segment-number, offset). The Segment Table stores the "Limit" (length) of each segment. If a process generates an address where the offset is greater than the limit, the hardware detects an illegal memory access and generates a trap (software interrupt). This prevents a process from accessing or overwriting data in other segments.

4. What is Thrashing? How is it controlled using the working set principle?

Thrashing occurs when a system's "page fault" rate is so high that the CPU spends more time swapping pages in and out of the disk than executing actual instructions.

• Working Set Principle: It defines the “Working Set” as the set of pages a process has referenced in the most recent \Delta time units. The OS tracks these sets and only allows a process to run if its entire working set fits in physical memory. If the total working sets of all processes exceed total RAM, the OS suspends a process to stop the thrashing.

5. What are the different states of a process?

A process changes states as it executes:
• New: The process is being created.
• Ready: The process is in memory, waiting for the CPU.
• Running: Instructions are being executed.
• Waiting (Blocked): The process is waiting for an event (like I/O completion).
• Terminated: The process has finished execution.

6. Differentiate between Internal fragmentation and External fragmentation.

• Internal Fragmentation: Occurs when memory is divided into fixed-size blocks. If a process is smaller than the block, the remaining space inside that block is wasted and cannot be used by others.
• External Fragmentation: Occurs when enough total free memory exists, but it is not contiguous (it’s scattered in small holes). Consequently, a large process cannot be loaded even though free space is available.

7. What is starvation of processes?

Starvation (Indefinite Blocking) happens when a process is ready to run but is never given the CPU because higher-priority processes keep arriving. This is a common flaw in Priority Scheduling. It is typically solved by Aging, where the priority of a process is gradually increased the longer it stays in the ready queue.

8. Differentiate between Multiprogramming, Multitasking, and Multiuser.

• Multiprogramming: Keeping multiple jobs in memory so that the CPU always has something to execute, maximizing CPU utilization.
• Multitasking: A logical extension where the CPU switches between multiple tasks (jobs) so frequently that users can interact with each program while it is running.
• Multiuser: Allows multiple users to access a single computer system simultaneously, often through remote terminals.

9. Define System Call.

A System Call is the programmatic way an application requests a service from the OS kernel. Since user programs run in “User Mode,” they cannot access hardware directly. They must use system calls (like open(), read(), or fork()) to switch to “Kernel Mode” so the OS can perform the task safely.

10. List the important responsibilities of an Operating System.

• Process Management: Scheduling, creation, and termination.
• Memory Management: Tracking and allocating RAM.
• File Management: Creating, deleting, and organizing files/directories.
• I/O Management: Buffering, caching, and managing device drivers.
• Protection/Security: Controlling access to system resources.

11. What is Cache Memory?

Cache Memory is a small, ultra-fast memory chip located close to the CPU. It holds copies of the data and instructions from the main memory (RAM) that are most likely to be used next. This reduces the average time to access data from the slower main memory.

12. Explain Storage Hierarchy.

Storage is organized based on speed and cost:
• Registers (Fastest, Smallest, Internal to CPU)
• Cache (L1, L2, L3)
• Main Memory (RAM)
• Secondary Storage (SSD, HDD)
• Tertiary Storage (Magnetic Tapes, Optical Disks)

13. Define Implicit tasking and Explicit tasking.

• Implicit Tasking: The compiler or runtime library automatically identifies and creates threads (e.g., OpenMP). The programmer does not manually manage the threads.
• Explicit Tasking: The programmer must explicitly use an API (like Pthreads or Win32 threads) to create, manage, and synchronize each thread in the code.

14. What is Compaction?

Compaction is a technique used to solve external fragmentation. The OS shifts all occupied memory blocks to one end of the physical memory, merging all small “holes” into one large, contiguous block of free memory. This requires that the system supports dynamic relocation.

15. Differentiate between Fixed size partition and Variable size partition.

• Fixed Partitioning: Memory is divided into static, permanent partitions. It is simple but causes Internal Fragmentation.
• Variable Partitioning: Memory is allocated dynamically based on the exact size of the process. It eliminates internal fragmentation but leads to External Fragmentation over time.

16. What is Multithreading?

Multithreading allows a single process to have multiple "threads" or paths of execution running concurrently. All threads within the same process share the same code and data but have their own registers and stacks, allowing for better performance in multi-core systems.

17. Differentiate between Preemptive and Non-preemptive scheduling.

• Preemptive: The OS can interrupt a running process and take the CPU away to give it to a higher-priority process (e.g., Round Robin).
• Non-preemptive: Once a process gets the CPU, it keeps it until it either terminates or switches to a waiting state (e.g., FCFS).

18. Differentiate between Logical address and Physical address.

• Logical Address: An address generated by the CPU during execution; it is "virtual" and doesn't exist in RAM.
• Physical Address: The actual location in the memory hardware. The Memory Management Unit (MMU) translates the logical address into a physical one at runtime.

19. List any three scheduling criteria and goals of scheduling algorithms.

• CPU Utilization: Keep the CPU as busy as possible (Goal: Maximize).
• Throughput: Number of processes completed per unit time (Goal: Maximize).
• Turnaround Time: The interval from submission to completion (Goal: Minimize).

20. What do you understand by Contiguous memory allocation?

In Contiguous Allocation, each process is assigned a single, continuous block of memory. The memory is usually divided into two partitions: one for the resident OS and one for user processes. This makes address translation simple but often leads to fragmentation.